“In The Name Of God”

Text Mining & Natural Language Processing

Assignment 1: Context-Aware Noisy Text Restoration with N-gram Language Models

Professor: Dr. Mostafa Fakhrahmad

Student: Paniz Azarnia 40435097

---

1 Problem Setting

Noisy text restoration is more challenging than traditional spell checking because errors cannot always be detected by examining words in isolation.

In conventional spell checking, a word that does not appear in the dictionary is assumed to be incorrect and is replaced with the closest valid word. However, in noisy text restoration, many errors involve words that are themselves valid dictionary entries but are incorrect within the sentence context.

For example:

- Noisy: I want too eat pizza
- Correct: I want to eat pizza

Both “too” and “to” are valid English words, so a simple dictionary-based spell checker cannot determine which one is correct. Resolving this type of ambiguity requires analyzing the surrounding context. Therefore, a language model that captures word sequence probabilities is necessary to determine which candidate fits best in a given sentence.

This project addresses the problem by combining candidate generation based on edit distance with context-aware scoring using an n-gram language model.

---

2 Method

The system processes noisy text through a multi-stage pipeline that transforms the input sentence into a corrected output sentence. The overall pipeline is as follows:

Noisy Sentence →

→ Tokenization

→ Candidate Generation

→ Candidate Scoring

→ Abstention Decision

→ Restored Sentence

First, each input sentence is tokenized using whitespace tokenization.

For every token, a set of candidate corrections is generated using edit distance (uses 1 but 2 if needed) operations and dictionary filtering.

Next, each candidate is scored using a combination of contextual probability from an n-gram language model, unigram prior probability, and an edit penalty.

The system then compares the score of the best candidate with the score of the original token.

If the score improvement exceeds a predefined threshold, the system replaces the token with the best candidate. Otherwise, it abstains and keeps the original token unchanged.

The corrected tokens are then combined to produce the final restored sentence.

---

3 Language Model Design

The system uses an n-gram language model to estimate the probability of word sequences and provide contextual information during restoration.

3.1 N-gram Orders

Three language model orders were evaluated: unigram, bigram, and trigram. Perplexity was measured on the development set to determine which model captured contextual dependencies most effectively. As expected, the trigram model produced the lowest perplexity and was therefore used in the final system.

3.2 Smoothing Method

The language model uses Kneser–Ney smoothing. This method applies absolute discounting and redistributes probability mass to unseen n-grams using continuation probabilities. Kneser–Ney smoothing is widely considered one of the most effective smoothing techniques for sparse n-gram models.

3.3 Tokenization

Sentences are tokenized using simple whitespace tokenization. Each sentence is split into tokens based on spaces.

3.4 Case Handling

The system applies flexible case handling by considering multiple case variants of candidate words, including lowercase, uppercase, and capitalized forms.

3.5 Unknown Word Strategy

Words that occur fewer than a specified threshold in the training corpus are mapped to a special unknown token (<UNK>). In this implementation, the unknown threshold is set to 1, meaning that rare words appearing only once are treated as unknown during language model training.

3.6 Perplexity Results

Perplexity was computed on the development set for unigram, bigram, and trigram models to evaluate how well each model predicts natural language sequences. The final tuning perplexity is as follows:

- Perplexity dev (order=1): 1137.1877
- Perplexity dev (order=2): 272.1816
- Perplexity dev (order=3): 198.7185

As expected, perplexity for trigram > bigram > unigram, which means trigram is the most confident in its answer, therefore why we chose trigram as our final model.

---

4 Restoration Design

4.1 Candidate Generation Strategy

Candidate are generated using edit distance operations applied to the vocabulary extracted from the training corpus. The system generates candidate words within edit distance one and two from the original token. These candidates are then filtered using the dictionary (the language model vocabulary) to ensure that only valid words are considered.

4.2 Scoring Function

Each candidate word is evaluated using a scoring function that combines contextual probability, unigram prior probability, and an edit penalty. The scoring function can be expressed as:

(As said in HW1 pdf) :

Final combined score

$$\text{score}_{\text{total}}(c) = \alpha \cdot \text{score}_{\text{context}}(c) + \beta \cdot \log P(c) + \gamma \cdot \text{edit_penalty}(c)$$

where:

- Context Score: represents the log probability of the candidate given the surrounding context from the language model. Both bigram and Trigram extensions were implemented and based on the final model were used. (In the current setting, the Trigram extension was used)

Basic contextual score (bigram)

$$\text{score}(c) = \log P(c \mid w_{i-1}) + \log P(w_{i+1} \mid c) \quad (3)$$

Trigram extension

$$\text{score}(c) = \log P(c \mid w_{i-2}, w_{i-1}) + \log P(w_{i+1} \mid w_{i-1}, c) + \log P(w_{i+2} \mid c, w_{i+1}) \quad (4)$$

- Unigram Probability: reflects the frequency of the candidate in the training data.
- Edit Penalty: penalizes candidates that require larger edit distances.

The weights used in the final system are:

- α = 1
- β = 1
- γ = −0.2

4.3 Abstain Rule

The system avoids unnecessary edits by applying an abstention rule. Let the score margin be defined as the difference between the score of the best candidate and the score of the original token:

margin = score(best_candidate) − score(original_token)

$$\text{score}(\hat{c}) - \text{score}(\text{original}) > \tau$$

If the margin exceeds a threshold τ, the token is replaced by the candidate. Otherwise, the system abstains and retains the original token.

4.4 Threshold Tuning Procedure

The threshold τ was tuned using the development set. Several candidate values were tested to balance correction recall and overcorrection. A value of τ = 0.02 was selected because it provided a good balance between correcting genuine errors and avoiding unnecessary edits.

Smoothing	N-gram	τ (tau)	γ (gamma)	Sentence Acc	Token Acc	WER	CER	Edit Precision	Edit Recall	Edit F1	Overcorrection
Add-k	Bigram	0.5	-1	0.2476	0.8959	0.0720	0.0126	0.9103	0.0771	0.1422	0.0064
Kneser-Ney	Bigram	0.5	-1	0.2508	0.8961	0.0718	0.0126	0.9043	0.0796	0.1462	0.0093
Kneser-Ney	Bigram	0.3	-1	0.2544	0.8966	0.0713	0.0125	0.9088	0.0839	0.1536	0.0088
Kneser-Ney	Trigram	0.2	-1	0.2896	0.9002	0.0678	0.0118	0.9473	0.1170	0.2083	0.0022
Kneser-Ney	Trigram	0.05	-0.5	0.3040	0.9018	0.0661	0.0116	0.9346	0.1320	0.2313	0.0019
Kneser-Ney	Trigram	0.03	-0.3	0.2668	0.8978	0.0701	0.0123	0.9383	0.0950	0.1726	0.0027
Kneser-Ney + punctuation	Trigram	0.05	-0.5	0.3040	0.9018	0.0661	0.0116	0.9346	0.1320	0.2313	0.0019
Kneser-Ney + punctuation	Trigram	0.03	-0.3	0.3100	0.9024	0.0655	0.0115	0.9301	0.1374	0.2394	0.0018
Kneser-Ney + punctuation	Trigram	**0.02**	**-0.2**	**0.3132**	**0.9027**	**0.0652**	**0.0114**	**0.9282**	**0.1404**	**0.2439**	**0.0018**

Between the last two rows, there's less than 0.005 of a change for F1, that's why we stopped.

---

5 Experimental Setting

5.1 Dataset

The experiments were conducted using the provided dataset, which consists of:

- train_clean.txt (used to train the language model)
- dev_noisy.txt (noisy sentences for evaluation and tuning)
- dev_clean.txt (reference sentences for development evaluation and tuning)
- test_noisy.txt (unlabeled noisy sentences for final prediction)

5.2 Training Procedure

The language model was trained on the clean training corpus. The development set was used to evaluate perplexity, tune hyperparameters, and analyze system performance.

5.3 Hyperparameters

- N-gram order: 3
- Unknown word threshold: 1
- α (context weight): 1
- β (unigram weight): 1
- γ (edit penalty weight): −0.2
- τ (abstention threshold): 0.02

This are already set and used in config.json .

5.4 Evaluation Metrics

The system was evaluated using several metrics, including sentence accuracy, token accuracy, word error rate (WER), character error rate (CER), edit precision, edit recall, edit F1 score, and overcorrection rate.

Smoothing	N-gram	τ (tau)	γ (gamma)	Sentence Acc	Token Acc	WER	CER	Edit Precision	Edit Recall	Edit F1	Overcorrection
Add-k	Bigram	0.5	-1	0.2476	0.8959	0.0720	0.0126	0.9103	0.0771	0.1422	0.0064
Kneser-Ney	Bigram	0.5	-1	0.2508	0.8961	0.0718	0.0126	0.9043	0.0796	0.1462	0.0093
Kneser-Ney	Bigram	0.3	-1	0.2544	0.8966	0.0713	0.0125	0.9088	0.0839	0.1536	0.0088
Kneser-Ney	Trigram	0.2	-1	0.2896	0.9002	0.0678	0.0118	0.9473	0.1170	0.2083	0.0022
Kneser-Ney	Trigram	0.05	-0.5	0.3040	0.9018	0.0661	0.0116	0.9346	0.1320	0.2313	0.0019
Kneser-Ney	Trigram	0.03	-0.3	0.2668	0.8978	0.0701	0.0123	0.9383	0.0950	0.1726	0.0027
Kneser-Ney + punctuation	Trigram	0.05	-0.5	0.3040	0.9018	0.0661	0.0116	0.9346	0.1320	0.2313	0.0019
Kneser-Ney + punctuation	Trigram	0.03	-0.3	0.3100	0.9024	0.0655	0.0115	0.9301	0.1374	0.2394	0.0018
Kneser-Ney + punctuation	Trigram	**0.02**	**-0.2**	**0.3132**	**0.9027**	**0.0652**	**0.0114**	**0.9282**	**0.1404**	**0.2439**	**0.0018**

---

6 Results

The evaluation results on the development set are reported in the dev_metrics.json file generated by the system. These metrics measure both correction accuracy and the system's tendency to introduce unnecessary edits.

The system's performance was also compared against three baseline approaches:

1. No-change baseline (returns the noisy input unchanged)
2. Unigram correction baseline (selects the candidate with the highest unigram probability, only works for OOV within edit distance 1, with no abstention threshold)
3. Context-aware n-gram system (the proposed method)

The proposed system achieved higher accuracy and better edit F1 scores than the baseline methods due to the use of contextual language modeling and the abstention mechanism.

Method	Sent Acc	Token Acc	WER	CER	Edit P	Edit R	Edit F1	Overcorr
No change	0.1756	0.8877	0.0804	0.0142	0.000	0.000	0.000	**0.000**
Unigram	0.3716	0.8923	0.0770	0.0139	0.4513	0.4350	0.4430	**0.407**
Context n-gram	0.3132	0.9027	0.0652	0.0114	0.9282	0.1404	0.2439	**0.0018**

The unigram baseline achieved the highest Edit F1 (0.443) and sentence accuracy (0.3716), mainly due to its aggressive correction strategy, which results in high recall (0.435). However, this comes at the cost of a very high overcorrection rate (0.407), meaning that many edits are incorrect.

In contrast, the proposed context-aware n-gram system achieves substantially lower WER (0.0652) and CER (0.0114), the highest token accuracy (0.9027), and an extremely low overcorrection rate (0.0018). Although its recall is lower (0.1404), the very high precision (0.9282) indicates that most edits are correct.

This demonstrates that the proposed system is significantly more conservative and reliable, achieving a better balance between correction quality and error introduction.

In practical applications, minimizing incorrect corrections is often more important than maximizing recall, making the proposed system preferable despite its lower F1 score.

---

7 Ablation Study

7.1 With vs. Without Edit Penalty (gamma =-0.2 vs 0)

Method	Sent Acc	Token Acc	WER	CER	Edit P	Edit R	Edit F1	Overcorr
With gamma	0.313	0.903	0.065	0.011	0.928	0.140	0.244	**0.002**
Without gamma	0.322	0.904	0.064	0.011	0.926	0.149	0.256	**0.003**

To study the effect of the edit penalty parameter (γ), we compared the system with γ =-0.2 and γ = 0. Removing the edit penalty slightly increases recall (0.140 → 0.149) and improves Edit F1 (0.244 → 0.256), as the system becomes more willing to apply corrections. This also results in a small improvement in sentence accuracy.

However, the absence of the penalty increases the overcorrection rate (0.002 → 0.003), indicating that the system performs more unnecessary edits. Introducing a negative edit penalty therefore makes the model more conservative, reducing incorrect corrections at the cost of slightly lower recall.

7.2 Bigram vs. trigram context scoring (ngram_order = 3 vs 2)

Method	Sent Acc	Token Acc	WER	CER	Edit P	Edit R	Edit F1	Overcorr
Trigram	0.3132	0.9027	0.0652	0.0114	0.9282	0.1404	0.2439	**0.0018**
Bigram	0.2796	0.8994	0.0685	0.0120	0.8921	0.1100	0.1958	**0.0088**

To assess the impact of context size, we compare bigram (n=2) and trigram (n=3) language models. The trigram model consistently outperforms the bigram model across all metrics. Edit F1 improves from 0.1958 to 0.2439, and the overcorrection rate drops from 0.0088 to 0.0018. This confirms that longer context provides more reliable probability estimates for correction decisions.

7.3 With vs. without dictionary filtering

To evaluate the effect of dictionary filtering, we commented out the vocabulary check inside get_candidate() in candidates.py:

```
# if word in self.vocab:
    # return (word,)
```

Method	Sent Acc	Token Acc	WER	CER	Edit P	Edit R	Edit F1	Overcorr
With filter	0.3132	0.9027	0.0652	0.0114	0.9282	0.1404	0.2439	**0.0018**
Without filter	0.0104	0.6921	0.2828	0.0489	0.0840	0.1936	0.1172	**0.8768**

Removing this constraint drastically degraded system performance. Precision dropped from 0.93 to 0.08 and the overcorrection rate increased from 0.0018 to 0.8768. This shows that dictionary-based pruning is essential for preventing spurious candidate generation, and without it the system becomes extremely aggressive and unstable.

---

8 Error Analysis

8.1 Wrong Edit:

No true overcorrections were observed in the development predictions. This suggests that the system is heavily regularized and prioritizes precision over recall.

Why can't we improve recall?
The recall remains relatively low despite removing the edit penalty and lowering the abstention threshold. This suggests that the main limitation lies in candidate generation rather than the scoring function. Since only edit-distance-1 candidates are considered, many true corrections are likely

absent from the candidate set. Expanding the candidate space (e.g., edit distance 2) could improve recall but would significantly increase computational cost.

8.2 Missed Real Error:

1. sentence_id 1

    - input: This allows them to move yp against gravity.
    - prediction: This allows them to move yp against gravity.
    - reference: This allows them to move up against gravity.
    - diagnosis: missed substitution (yp → up)

2. sentence_id 2

    - input: '''Valencia C.F.''' is a football team n Spain.
    - prediction: '''Valencia C.F.''' is a football team n Spain.
    - reference: '''Valencia C.F.''' is a football team in Spain.
    - diagnosis: missed preposition insertion (n → in)

8.3 Interesting Case:

1. sentence_id 55

    - input: It was not great music, but the amazing thing was that tere was a composer oif genius around.
    - prediction: It was not great music, but the amazing thing was that tere was a composer of genius around.
    - reference: It was not great music, but the amazing thing was that there was a composer of genius around.
    - diagnosis: partial correction — fixed oif → of but missed tere → there

2. sentence_id 101

    - input: 80% of the population ot Iraniuan Armenia were Muslims.
    - prediction: 80% of the population of Iraniuan Armenia were Muslims.
    - reference: 80% of the population of Iranian Armenia were Muslims.
    - diagnosis: partial correction — fixed ot → of but missed Iraniuan → Irainian

3. sentence_id 190

    - input: If is sometimes uszed instead of 'Lord'.
    - prediction: If is sometimes used instead of 'Lord'.
    - reference: It is sometimes used instead of 'Lord'.
    - diagnosis: partial correction — fixed uszed → used but missed If → It

9 Reflection

9.1 Systematic Weakness

A systematic weakness of the current system is its overly conservative nature, which leads to low recall. While this design choice effectively minimizes overcorrection (achieving near-zero overcorrection rate), it results in a significant number of true errors being missed, as indicated by the recall score of 0.140.

9.2 Improvement with More Time

With more time, a concrete improvement would be to expand the candidate generation mechanism. Specifically, incorporating candidates with an edit distance of 2 (in addition to the current edit distance 1) would significantly increase the pool of potential corrections. This is expected to directly address the low recall by ensuring that more of the true errors are represented in the system's candidate set, thus allowing the scorer to identify and select the correct form.

10 AI Usage Disclosure

AI tools were used during the development of this project for debugging, code clarification, and report writing assistance. ChatGPT-5.5 (via the OpenAI API) was used to help identify issues in the smoothing implementation, clarify scoring logic, and assist in drafting parts of the report. All core design decisions, implementation adjustments, and experimental evaluations were performed by the author.